



PHARMADASH

Technical Documentation

pharmacy_updated.py — v2.0

Pharmacy Inventory & Sales Analytics Platform

Multi-Store Dashboard for Hyderabad Pharmacy Chains

Document Version	v2.0 — Updated Release
Date	May 2026
Platform	Streamlit (Python 3.11)
Stores Covered	5 Hyderabad Locations
Lines of Code	3,083 lines (+52% from v1)
AI Engine	Groq + Llama-3.3-70B

Table of Contents

- Table of Contents 2
 - 1.1 Project Overview 4
 - 1.2 Business Objectives 4
 - 1.3 Technology Stack..... 4
 - 1.4 Stores Covered 4
 - 2.1 High-Level Data Flow 5
 - 2.2 Module Structure 5
 - 2.3 Caching Strategy..... 5
 - 3.1 Fact_Sales_20k.csv 6
 - 3.2 Fact_Stock_20k.csv 6
 - 3.3 Dimension Tables..... 7
 - Dim_Products_20k.csv 7
 - Dim_Store.csv 7
 - Dim_Suppliers_20k.csv 7
 - 3.4 Key Joins..... 7
 - 4.1 load_data() Function 8
 - File Validation 8
 - Error Handling 8
 - 4.2 Dtype Optimisations 8
 - 4.3 Category Dtype for String Columns 8
 - 4.4 Vectorised KPI Computation..... 8
 - 5.1 Filter Functions..... 9
 - get_filtered_sales(dr_min, dr_max, sel_store, sel_cat) 9
 - get_filtered_stock(sel_store, sel_cat, sel_supplier, sel_status) 9
 - 5.2 KPI Functions..... 9
 - get_overview_kpis(...)..... 9
 - get_safety_stock_kpis(...) 9
 - 5.3 Aggregation Functions..... 9
 - 5.4 Bug-Fix Functions (New in v2.0)..... 10
 - get_expiring_soon_n(days, ...)..... 10
 - get_out_of_stock_by_store(...)..... 10
 - get_supplier_last_delivery_days(...)..... 10
 - get_reorder_list(...) 10
 - 6.1 Sidebar — Global Filter Panel 11
 - 6.2 Page 1 — Home..... 11
 - 6.3 Page 2 — Overview 11
 - 6.4 Page 3 — Inventory & Expiry..... 11
 - 6.5 Page 4 — Sales & Demand..... 11
 - 6.6 Page 5 — Supplier & Store 12
 - 6.7 Page 6 — Business Insights..... 12
 - 6.8 Page 7 — AI Assistant..... 12
 - 6.9 Page 8 — About..... 12
 - 7.1 Client Initialisation 13
 - 7.2 Context Building (build_ai_context) 13
 - 7.3 generate_ai_insights() Signature (v2)..... 13
 - 7.4 System Prompt..... 14
 - 7.5 Caching Strategy for AI 14
 - 8.1 Design Philosophy..... 15
 - 8.2 CSS Variables (Colour Tokens)..... 15
 - 8.3 Typography 15
 - 8.4 Animated Background 15
 - 8.5 Key Component Classes 16
 - 9.0 Version Comparison(v1→v2)

- 10.1 Prerequisites 19
- 10.2 Python Dependencies 19
- 10.3 Environment Setup..... 19
- 10.4 Dataset Configuration..... 19
- 10.5 Running the Application..... 19
- 10.6 Required CSV Files..... 19
- 11.1 Hardcoded Dataset Path 20
- 11.2 Product Name Dataset Limitation 20
- 11.3 GST and Net Profit Estimates..... 20
- 11.4 AI API Availability 20
- 11.5 Recommended Improvements..... 20
- 12.Glossary

1. Introduction

Project overview, goals, and technology stack

1.1 Project Overview

PharmaDash is a full-stack pharmacy analytics platform built with Python and Streamlit. It is designed to help multi-store pharmacy chains in Hyderabad make smarter, faster, data-driven decisions across inventory management, sales tracking, supplier evaluation, and business strategy.

The application connects five data sources (sales transactions, stock records, product master, store data, and supplier data) and presents them through seven interactive dashboard pages. Every page responds to a shared sidebar filter panel so that all charts and KPIs stay in sync as the user drills down.

1.2 Business Objectives

- Centralise inventory and sales data across 5 Hyderabad pharmacy stores into a single web dashboard.
- Reduce medicine wastage caused by expiry and overstocking through real-time expiry tracking.
- Enable proactive reorder and procurement decisions with automatic low-stock alerts and suggested order quantities.
- Surface revenue trends, top-performing SKUs, and margin insights instantly for management decision-making.
- Provide supplier scorecards to improve lead times, detect delivery delays, and flag quality issues.
- Deliver AI-powered Q&A via a Groq-backed Llama model so staff can ask natural-language questions about pharmacy data.

1.3 Technology Stack

Python 3.11	Core application language
Streamlit	Interactive web dashboard framework
Pandas + NumPy	Data processing and vectorised operations
Plotly Express / GO	Interactive charts, maps, and visualisations
OpenStreetMap (Plotly)	Store geo-visualisation with revenue bubbles
openpyxl	Excel report export from filtered data
Groq (Llama-3.3-70B)	AI-powered pharmacy assistant & multi-turn chat
python-dotenv	Secure API key loading from .env file

1.4 Stores Covered

MedPlus	Ameerpet, Hyderabad Lat: 17.4344, Lon: 78.4487
Apollo	Kukatpally, Hyderabad Lat: 17.4924, Lon: 78.3918
WellCare	Banjara Hills, Hyderabad Lat: 17.4138, Lon: 78.4349
HealthFirst	Madhapur, Hyderabad Lat: 17.4474, Lon: 78.3910
CityMed	Dilsukhnagar, Hyderabad Lat: 17.3688, Lon: 78.5268

2. Application Architecture

Data flow, module layout, and component structure

2.1 High-Level Data Flow

The application follows a layered architecture with three distinct tiers:

1. Data Layer — five CSV files are loaded once at server startup via `@st.cache_data`. They are never mutated; all downstream operations create filtered views.
2. Logic Layer — a library of `@st.cache_data` functions accept filter parameters as arguments and return computed DataFrames or dictionaries. Streamlit caches results keyed by parameter hash, so repeated filter selections cost zero computation.
3. Presentation Layer — seven page modules render charts, tables, and KPI cards using Plotly Express, Plotly Graph Objects, and custom HTML/CSS injected via `st.markdown`.

2.2 Module Structure

Lines 1–45	Imports, Groq client initialisation, page config
Lines 46–382	CSS — all styles inline (glassmorphism, sidebar, KPI cards, AI chat bubbles)
Lines 386–420	Module-level constants: CAT_PALETTE, BLUES, STORE_COORDS, filter option lists
Lines 422–516	<code>load_data()</code> — CSV loading, joins, dtype optimisations, filter list population
Lines 525–900	Cached aggregation functions — 20+ <code>@st.cache_data</code> helpers
Lines 900–1120	Bug-fix functions: <code>get_expiring_soon_n</code> , <code>get_out_of_stock_by_store</code> , <code>get_supplier_last_delivery_days</code> , <code>get_reorder_list</code>
Lines 1120–1300	Sidebar — filter form, date range, download button
Lines 1300–1500	Page 1: Home
Lines 1500–1700	Page 2: Overview (KPIs + charts)
Lines 1700–1900	Page 3: Inventory & Expiry
Lines 1900–2100	Page 4: Sales & Demand
Lines 2100–2500	Page 5: Supplier & Store
Lines 2500–2700	Page 6: Business Insights
Lines 2700–2960	Page 7: AI Assistant
Lines 2960–3083	Page 8: About

2.3 Caching Strategy

The application uses `@st.cache_data` throughout to avoid redundant computation on every Streamlit rerun. The caching model works as follows:

- `load_data()` is cached for the lifetime of the server process. CSVs are read from disk exactly once.
- Every aggregation function (`get_filtered_sales`, `get_overview_kpis`, `get_monthly_revenue`, etc.) is cached by its scalar filter parameters. When the user changes a filter the cache key changes, forcing a recompute. If the user returns to the same filter combination, the result is served from cache instantly.
- AI insight generation (`generate_ai_insights`) is cached by (summary_data, user_question, chat_history) tuple. Identical questions on the same data cost zero API calls.
- `build_report()` (Excel export) is cached similarly so the Excel file is rebuilt only when filters change.

3. Data Model

CSV schema, field descriptions, and data relationships

3.1 Fact_Sales_20k.csv

20,000 sales transaction records. Each row represents a single product sale at a store on a specific date.

TransactionID	Unique transaction identifier (string)
Date	Transaction date — parsed to datetime, dayfirst=False
Store_ID	Foreign key joining to Dim_Store.csv
ProductID	Foreign key joining to Dim_Products_20k.csv
Medicine_Name	Medicine trade name (category dtype for performance)
Category	Therapeutic category: Analgesics, Antibiotics, Diabetes, Vitamins, Cardiac, Antiseptics
QuantitySold	Units sold — downcasted to float32
UnitPrice	Selling price per unit
TotalAmount	QuantitySold × UnitPrice — primary revenue metric
CustomerType	Hospital or Retail
Month / Month_Num	Derived columns added after loading for groupby performance

3.2 Fact_Stock_20k.csv

20,000 stock batch records. Each row represents a single product batch at a store.

Store_ID	Foreign key joining to Dim_Store.csv
ProductID	Foreign key joining to Dim_Products_20k.csv
SupplierID	Foreign key joining to Dim_Suppliers_20k.csv
BatchNumber	Batch identifier (category dtype)
ExpiryDate	Date the batch expires — used to compute DaysToExpiry
DaysToExpiry	Days until expiry — numeric, float32
ExpiryStatus	Valid / Expired / Near Expiry (category dtype)
QuantityOnHand	Current stock level — float32
ReorderPoint	Threshold below which reorder is triggered — float32
SafetyStock	Minimum buffer stock level — float32
UnitCost	Procurement cost per unit — float32

3.3 Dimension Tables

Dim_Products_20k.csv

ProductID	Primary key
ProductName	Product display name (170 unique IDs, 6 name templates)
Category	Therapeutic category
UnitCost	Purchase cost
RetailPrice	Suggested retail price
ReorderPoint	Reorder threshold
SafetyStock	Safety buffer level

Dim_Store.csv

Store_ID	Primary key
Store_Name	Full store name (e.g. MedPlus Ameerpet)

Note: Latitude and longitude for map visualisation are maintained in the STORE_COORDS dictionary in code, not in the CSV.

Dim_Suppliers_20k.csv

SupplierID	Primary key
Supplier_Name	Supplier company name
QualityRating	Rating 1–5 (float) — used in risk scoring
LeadTimeDays	Average delivery lead time in days

3.4 Key Joins

All joins are performed once inside `load_data()` and the enriched DataFrames are stored as module-level globals:

- `sales_df` ← `Fact_Sales` + `Store_Name` (via `Store_ID`) + `UnitCost` (via `ProductID`)
- `stock_df` ← `Fact_Stock` + `Store_Name` (via `Store_ID`) + `ProductName`, `Category`, `UnitCost`, `RetailPrice`, `ReorderPoint`, `SafetyStock` (via `ProductID`) + `Supplier_Name`, `QualityRating`, `LeadTimeDays` (via `SupplierID`)

4. Data Loading & Performance Optimisations

Efficient loading, dtype strategies, and caching details

4.1 load_data() Function

The `load_data()` function is the single entry point for all CSV data. Decorated with `@st.cache_data`, it executes only once per server process lifetime — all subsequent calls receive the cached result.

File Validation

Before reading any file, the function iterates over all five required CSVs and checks `os.path.exists()`. If any file is missing, it calls `st.error()` with a descriptive message showing the expected path, then `st.stop()` to halt rendering cleanly. This prevents cryptic `FileNotFoundError` tracebacks from appearing in the UI.

Error Handling

Three exception layers protect against failures:

- `st.runtime.scriptrunner.StopException` — re-raised immediately so `st.stop()` can propagate correctly up the call stack.
- `FileNotFoundError` — caught separately to show a targeted error message about the dataset folder.
- `Exception (generic)` — last resort that surfaces the raw exception message for debugging.

4.2 Dtype Optimisations

After loading, numeric columns are downcasted from the default `float64` to `float32`, halving memory usage for large numeric arrays. The updated code uses the Pandas 2.x-safe pattern:

```
# Pandas 2.x safe - downcast= parameter is deprecated
sales[col] = pd.to_numeric(sales[col], errors="coerce").astype("float32")

# Applied to: QuantitySold, TotalAmount, Month_Num, UnitCost (sales)
# Applied to: QuantityOnHand, UnitCost, RetailPrice, ReorderPoint,
#             SafetyStock, QualityRating, LeadTimeDays, DaysToExpiry (stock)
```

4.3 Category Dtype for String Columns

High-cardinality string columns (`Category`, `Store_Name`, `Supplier_Name`, `ProductName`, etc.) are converted to Pandas category dtype. This provides two benefits:

- **Memory:** category dtype stores each unique value once as an integer code, dramatically reducing RAM for repeated values (e.g. 6 categories across 20,000 rows).
- **Speed:** groupby operations on category columns are significantly faster because Pandas avoids hash-map lookups on raw strings.

4.4 Vectorised KPI Computation

KPI calculations use NumPy vectorised operations instead of row-wise `DataFrame.apply()`. For example, total inventory value uses `np.dot(qty, cost)` — a single BLAS-optimised dot product — rather than `(df["qty"] * df["cost"]).sum()`, which creates a temporary Series.

```
# Vectorised inventory value (no temporary Series created)
qty = fk["QuantityOnHand"].to_numpy()
cost = fk["UnitCost"].to_numpy()
total_sv = float(np.dot(qty, cost))

# Vectorised stock status (no row-wise apply)
df["StockStatus"] = np.select(
    [qty == 0, qty < rp],
    ["Out of Stock", "Low Stock"],
    default="In Stock"
)
```

5. Aggregation & Helper Functions

Complete reference for all `@st.cache_data` functions

5.1 Filter Functions

`get_filtered_sales(dr_min, dr_max, sel_store, sel_cat)`

Returns a filtered view of `sales_df` applying date range, optional store, and optional category filters. Returns the original reference (no copy) unless a filter is applied.

`get_filtered_stock(sel_store, sel_cat, sel_supplier, sel_status)`

Returns a filtered view of `stock_df`. The `sel_status` parameter maps to:

"Low Stock"	QuantityOnHand > 0 AND QuantityOnHand < ReorderPoint
"Out of Stock"	QuantityOnHand == 0
"Available"	QuantityOnHand >= ReorderPoint
"All"	No stock status filter applied

5.2 KPI Functions

`get_overview_kpis(...)`

Returns a dict with: `total_sv` (inventory value), `total_rev` (sales revenue), `total_units` (units sold), `expired_cnt` (expired batches), `low_stock` (low-stock batches), `safe_pct` (% batches with Valid status). All computed via NumPy vectorised ops.

`get_safety_stock_kpis(...)`

Returns `below_safety` (items below SafetyStock), `below_reorder` (items below ReorderPoint), `out_of_stock` (items with zero quantity). Used as supplementary context for the AI assistant.

5.3 Aggregation Functions

<code>get_monthly_revenue</code>	Revenue grouped by <code>Month_Num + Month</code> — sorted chronologically for the trend line chart.
<code>get_category_revenue</code>	Revenue summed by <code>Category</code> — used in the donut/pie chart on the Overview page.
<code>get_top_medicines</code>	Top N medicines by <code>TotalAmount</code> — used in the horizontal bar chart.
<code>get_batch_expiry_table</code>	Full batch expiry detail DataFrame with vectorised <code>StockStatus</code> column.
<code>get_store_performance</code>	Revenue, UnitsSold, Transactions, <code>Stock_OnHand</code> per store. String-formatted with vectorised <code>map()</code> .
<code>get_top15_medicines</code>	Top 15 medicines by revenue with formatted Revenue and Units columns.
<code>get_category_stock_health</code>	Per-category Safe/LowStock counts via vectorised one-hot approach (avoids <code>groupby+apply</code> lambda).
<code>get_margin_by_category</code>	Average $(UnitPrice - UnitCost) / UnitPrice$ margin % per category. Flags negative margins.
<code>get_customer_type_revenue</code>	Revenue split by <code>CustomerType</code> (Hospital vs Retail).
<code>get_medicine_velocity</code>	Fast vs Slow Moving classification based on units/day vs dataset median. Used in Sales page.

get_map_data	Store revenue + lat/lon + bubble size for Plotly scatter_mapbox. Lat/lon cast to float64 to avoid Plotly mean() TypeError on category dtype.
build_report	Builds Excel binary (openpyxl) from filtered Sales + Stock data for download.

5.4 Bug-Fix Functions (New in v2.0)

get_expiring_soon_n(days, ...)

Previous version used `ProductName.nunique()` which returned only 6 (the dataset has 6 medicine name templates shared across 170 unique ProductIDs). Fixed to return four metrics:

- `batches` — number of stock rows expiring within the window (len of filtered df)
- `skus` — unique ProductID count (accurate SKU-level count)
- `med_names` — unique ProductName count (display-level)
- `total_qty` — total QuantityOnHand at risk of expiry

get_out_of_stock_by_store(...)

Previous version called `drop_duplicates` on `[Store_Name, ProductName]`, which collapsed all ProductIDs sharing the same name — resulting in a maximum of 30 rows (6 names × 5 stores) instead of the real 93+ unique out-of-stock SKU+store combinations. Fixed to dedup on `[Store_Name, ProductID]`.

get_supplier_last_delivery_days(...)

Completely reimplemented. Previous version sorted suppliers purely by `EstDaysSinceDelivery` (days since estimated last delivery), which surfaced suppliers with zero OOS batches while hiding suppliers actively causing stockouts. New composite RiskScore formula:

```
RiskScore = OOSBatches      × 10  # dominant: supplier causing real stockouts
            + LowStockBatches × 3  # urgent: about to go OOS
            + ExpiringBatches × 1  # stock at risk of write-off
            + EstDaysSinceDelivery × 0.3
            + LeadTimeDays    × 0.5
            + (5 - QualityRating) × 2 # chronic quality risk
```

get_reorder_list(...)

New helper that identifies all stock items below their `ReorderPoint` and computes a suggested order quantity:

```
SuggestedOrder = max(1, round(ReorderPoint × 1.5 - QuantityOnHand))
```

Returns a DataFrame with `ProductName`, `Store_Name`, `Supplier_Name`, `QuantityOnHand`, `ReorderPoint`, `SafetyStock`, `UnitCost`, `Category`, `SuggestedOrder`.

6. Dashboard Pages

Per-page feature descriptions and component inventory

6.1 Sidebar — Global Filter Panel

All pages share a single sidebar rendered before the page router. The sidebar contains:

- Navigation — radio buttons styled as pill-shaped nav items for page selection.
- Date Range — two `date_input` widgets (start / end date) defaulting to the full dataset range.
- Store — selectbox with "All" + 5 store options.
- Category — selectbox with "All" + 6 therapeutic categories.
- Supplier — selectbox with "All" + all supplier names.
- Stock Status — selectbox: All / Low Stock / Out of Stock / Available.
- Apply Filters button — `st.form_submit_button` that triggers a rerun.
- Clear Filters button — resets all `session_state` filter keys and reruns.
- Download Filtered Report — `st.download_button` serving the Excel binary from `build_report()`.

6.2 Page 1 — Home

Landing page presenting the platform identity, a live KPI snapshot (4 headline metrics), and a feature overview grid. Designed for executives who need an at-a-glance summary. No detailed charts — routes users to the appropriate page.

6.3 Page 2 — Overview

KPI dashboard with the following components:

- KPI row: Total Revenue, Total Units Sold, Inventory Value, Expired Batches, Low-Stock Batches, Stock Health %.
- New in v2: Net Profit estimate (Revenue – COGS – Wastage), GST liability estimate (12% of revenue), Rx vs OTC proxy split.
- Monthly Revenue trend line (Plotly Express line chart).
- Category Revenue donut chart.
- Store performance table.
- Category stock health stacked bar (Safe vs Low Stock per category).

6.4 Page 3 — Inventory & Expiry

Batch-level expiry management page:

- Summary KPIs: batches expiring in 7 days (accurate counts via `get_expiring_soon_n`), total expired, out-of-stock SKUs (accurate via `get_out_of_stock_by_store`).
- Interactive expiry table with conditional row colouring (Expired = red, Near Expiry = amber, Valid = green).
- Expiry status distribution bar chart.
- Days-to-expiry histogram showing the expiry risk timeline.

6.5 Page 4 — Sales & Demand

Sales analytics with trend and product-level granularity:

- Monthly revenue trend (line chart) with year-over-year context.
- Top-10 medicines by revenue (horizontal bar chart with category colour coding).
- Category revenue split (bar chart).

- New in v2: Rx vs OTC proxy split (Customer Type — Hospital treated as Rx proxy).
- Medicine velocity table: Fast Moving vs Slow Moving segmentation based on units/day vs dataset median.
- Top-15 medicines table with formatted revenue and unit columns.

6.6 Page 5 — Supplier & Store

Geographical and supplier performance page:

- Plotly scatter_mapbox (OpenStreetMap) with bubble size proportional to store revenue and hover showing store name, revenue, units sold.
- Store performance table (Revenue, UnitsSold, Transactions, Stock_OnHand).
- Supplier risk table: RiskScore-ranked top-5 suppliers with OOSBatches, LowStockBatches, LeadTimeDays, QualityRating columns.
- Supplier quality scatter plot (QualityRating vs LeadTimeDays).

6.7 Page 6 — Business Insights

Consolidated operational recommendations page — new in v2:

- Store underperformance alerts: stores with revenue below the median across the selected period flagged with amber callout cards.
- Below-cost category flags: categories where average margin % is negative (selling at a loss) — critical alert with red callout.
- Reorder list: full table from `get_reorder_list()` showing products below reorder point with suggested order quantities and estimated reorder cost.
- AI-generated summary: calls `generate_ai_insights` with enriched context including below-cost, safety stock KPIs, and profitability estimate.

6.8 Page 7 — AI Assistant

Groq-powered conversational assistant with multi-turn chat — major redesign in v2:

- Pre-built insight cards: the page auto-calls `generate_ai_insights` on load to produce 4 Q&A cards covering reorder needs, supplier performance, expiry risk, and revenue improvement.
- Multi-turn chat: chat history is stored in `st.session_state["ai_chat_history"]` and passed as a tuple to `generate_ai_insights`. Prior turns are replayed as user/assistant message pairs before the new question.
- Ask button (primary) and Clear Chat button (secondary) in a two-column layout.
- Bubble-style chat rendering: user messages in right-aligned blue bubbles, assistant messages in left-aligned white bubbles.

6.9 Page 8 — About

Documentation page showing platform purpose, data sources, stores covered, technology stack, and key features. The Performance Optimisations section moved to full-width with CSS `column-count:2` layout.

7. AI Integration

Groq + Llama-3.3-70B — context building, multi-turn chat, and caching

7.1 Client Initialisation

The Groq client is initialised once at module load inside a try/except block. If the groq package is not installed or the GROQ_API_KEY environment variable is not set, _groq_client is set to None and all AI features gracefully degrade to displaying a "not configured" message.

```
_groq_client = None
if _GROQ_AVAILABLE:
    _api_key = os.getenv("GROQ_API_KEY", "")
    if _api_key:
        _groq_client = Groq(api_key=_api_key)
```

7.2 Context Building (build_ai_context)

The build_ai_context() function assembles a structured text summary of the current filtered state, passed as the summary_data argument to generate_ai_insights. In v2 this context was significantly enriched:

- Basic KPIs: total revenue, units, inventory value, expired batches, low stock batches.
- Category revenue breakdown.
- Supplier risk table.
- Below-cost categories: categories where average MarginPct < 0, flagged with "ACTION REQUIRED" label.
- Safety stock KPIs: below_safety, below_reorder, out_of_stock counts.
- Profitability estimate: Total Revenue, estimated COGS (QuantitySold × UnitCost), Wastage, Net Profit, GST at 12%.

7.3 generate_ai_insights() Signature (v2)

```
@st.cache_data(show_spinner=False)
def generate_ai_insights(
    summary_data: str,
    user_question: str | None = None,
    chat_history: tuple = () # NEW in v2: tuple of alternating user/assistant
    strings
) -> str:
```

The chat_history tuple is structured as (user_msg_1, assistant_msg_1, user_msg_2, assistant_msg_2, ...). The function replays prior turns before the current user message:

```
_messages = [{"role": "system", "content": system_prompt}]
for _u, _a in zip(chat_history[:2], chat_history[1::2]):
    _messages.append({"role": "user", "content": _u})
    _messages.append({"role": "assistant", "content": _a})
_messages.append({"role": "user", "content": user_content})
```

7.4 System Prompt

The system prompt instructs the model to act as a senior pharmacy analytics expert, use INR currency formatting, and respond with structured Q&A pairs in the format "Q: ... A: ..." with bullet-pointed action items. This format is then parsed by the front-end to render the styled Q&A card grid.

7.5 Caching Strategy for AI

`generate_ai_insights` is decorated with `@st.cache_data`. The function is cached by the combination of `(summary_data, user_question, chat_history)`. This means:

- If the filters have not changed, the same context string is produced and the AI result is served from cache — zero API calls.
- If the user asks the same question twice in the same session with the same filters, the cached answer is returned immediately.
- Changing any filter invalidates the cache for that parameter combination, triggering a fresh API call with updated context.

8. CSS & UI Design System

Glassmorphism design, colour tokens, and component styles

8.1 Design Philosophy

PharmaDash uses a glassmorphism design language — cards and panels appear as frosted glass floating above a soft blue medical-themed gradient background. The design is implemented entirely via inline CSS injected with `st.markdown(unsafe_allow_html=True)`.

8.2 CSS Variables (Colour Tokens)

<code>--brand</code>	#1565C0 — Primary blue, used for headings, borders, accents
<code>--brand-lt</code>	#42A5F5 — Light blue, used for secondary accents
<code>--accent</code>	#00BCD4 — Teal, used for the AI chat card border
<code>--danger</code>	#E53935 — Red, used for expired/critical alerts
<code>--warn</code>	#FB8C00 — Amber, used for near-expiry / low-stock warnings
<code>--ok</code>	#43A047 — Green, used for in-stock / healthy status indicators
<code>--txt</code>	#0d2f52 — Dark navy, default body text colour

8.3 Typography

Two Google Fonts are imported: Nunito (body text, weights 300–900) and Rajdhani (headings, KPI values, section titles, weights 500–700). Nunito provides friendly readability for data-dense tables; Rajdhani provides a technical, bold appearance for metric values.

8.4 Animated Background

The `.stApp` background uses a 6-layer radial gradient (soft blue orbs at different positions and opacities) over a linear-gradient base, creating depth without images. Two pseudo-elements add:

- `.stApp::before` — a tiled SVG pattern of hexagons and medical crosses that slowly drifts via the `hexdrift` CSS animation (35s infinite alternate).
- `.stApp::after` — a large radial gradient orb in the top-right corner that moves and scales via the `orb` CSS animation (22s infinite alternate).

8.5 Key Component Classes

.kpi-card	Glassmorphism metric card with gradient background, border-radius 16px, hover lift effect. Variants: <code>.warn</code> (red/amber), <code>.ok</code> (green), <code>.teal</code> .
.insight-box	White semi-transparent card (rgba 0.72) for section content, used on About and Insights pages.
.page-title	Rajdhani font, centred, text-shadow for legibility over the animated background.
.sec-title	Left-bordered section heading bar (4px solid #42A5F5) with rounded right corners.
.ai-response-card	Blue left-bordered card for AI auto-insight results.
.ai-chat-card	Teal left-bordered card for AI chat answers (v1 legacy).
.qa-card	Two-column grid card: left cell = blue gradient question label, right cell = white answer area.
.qa-problem / .qa-solution	Red/green bordered callout blocks inside Q&A answers.

Performance Note — backdrop-filter Removal

backdrop-filter:blur() was removed from insight-box, KPI panels, and AI cards in v2. The blur filter triggers GPU compositing on every rerender, causing visible jank on lower-end hardware. The glassmorphism effect is maintained via semi-transparent backgrounds and box-shadow without the blur cost.

9. Version Comparison (v1 → v2)

Detailed diff of all changes between pharmacy.py and pharmacy_updated.py

The table below summarises all significant changes between the original pharmacy.py (2,031 lines) and pharmacy_updated.py (3,083 lines, +52%). Changes are categorised as Added (new functionality), Changed (modified behaviour), or Removed (deleted code).

Area	Type	Before (pharmacy.py)	After (pharmacy_updated.py)
File Size	Changed	2,031 lines	3,083 lines (+1,052 lines, +52%)
CSS Comment	Changed	# lightweight glassmorphism, no blur/animation	# all styles inline (no external file needed)
Block Container Padding	Changed	padding-top:1.2rem	padding-top:1.4rem; + bottom/left/right 2rem
KPI Row Gap	Changed	gap:11px; margin-bottom:18px	gap:14px; margin-bottom:22px
Streamlit Column Gap	Added	—	stHorizontalBlock{gap:1.2rem} + stColumn{gap:0}
Streamlit Vertical Gap Reset	Added	—	.stMarkdown{margin-bottom:0}, stVerticalBlock>div{gap:0.55rem}
stMetric Styling	Added	—	Background, border-radius, padding, box-shadow on st.metric elements
backdrop-filter blur	Removed	Present on insight-box, AI cards, KPI panels	Removed for render performance improvement
Data Loading Path	Changed	os.path.dirname(__file__) (relative)	Hardcoded absolute Windows path
File Validation	Added	—	Pre-flight check for all 5 CSVs with st.error + st.stop
Error Handling	Added	No try/except	StopException, FileNotFoundError, Exception handlers
pd.to_numeric Downcast	Changed	downcast="float" (deprecated)	errors="coerce" + .astype("float32") — Pandas 2.x safe
AI: chat_history param	Added	Single-turn only	chat_history: tuple for multi-turn context replay
AI: Multi-turn messages	Added	system + user only	Prior turns replayed as user/assistant pairs
AI Chat UI	Changed	Single input + 1 Ask button	Ask (primary) + Clear Chat + scrolling bubble chat history
AI Context: Below-Cost	Added	—	Categories with MarginPct<0 flagged in AI context
AI Context: Safety Stock	Added	—	below_safety, below_reorder, out_of_stock in context
AI Context: Profitability	Added	—	Revenue, COGS, Wastage, Net Profit, GST estimate
get_expiring_soon_n	Added	Used ProductName.nunique()→6 wrong	Returns batches, skus, med_names, total_qty correctly

get_out_of_stock_by_store	Added	Dedup on ProductName→max 30 rows	Dedup on ProductID→93+ accurate OOS rows
get_supplier_last_delivery_days	Changed	Sort by EstDaysSinceDelivery only	Composite RiskScore: OOS×10 + LowStock×3 + Expiring×1 + delay×0.3 + lead×0.5 + quality×2
get_reorder_list()	Added	—	New helper: items below ReorderPoint + SuggestedOrder
Overview Page KPIs	Added	Revenue, units, expiry, health %	Added Net Profit, GST estimate, Rx vs OTC proxy
Business Insights Page	Added	—	Underperformance alerts, below-cost flags, reorder list
About: Perf. Optimisations	Changed	Inside c2 column (narrow)	Full-width with column-count:2 CSS

10. Installation & Configuration

Setup guide, environment variables, and running the application

10.1 Prerequisites

- Python 3.11 or later
- pip (Python package manager)
- A Groq API key (free tier available at console.groq.com)
- The five dataset CSV files in the configured directory

10.2 Python Dependencies

```
pip install streamlit pandas numpy plotly openpyxl groq python-dotenv
```

10.3 Environment Setup

Create a .env file in the same directory as pharmacy_updated.py:

```
# .env
GROQ_API_KEY=gsk_your_api_key_here
```

10.4 Dataset Configuration

The current code has a hardcoded dataset path:

```
base = r"C:\Users\Asus\OneDrive\Desktop\pharma_dash"
```

Important

This path must be updated to match the deployment environment.
For production, replace with: `base = os.path.join(os.path.dirname(os.path.abspath(__file__)), "dataset")`
Then place all five CSV files in a "dataset" subdirectory next to the script.

10.5 Running the Application

```
# Navigate to the project directory
cd C:\Users\Asus\OneDrive\Desktop\pharma_dash

# Run Streamlit
streamlit run pharmacy_updated.py

# The app opens at http://localhost:8501
```

10.6 Required CSV Files

Fact_Sales_20k.csv	20,000 sales transactions
Fact_Stock_20k.csv	20,000 stock batch records
Dim_Products_20k.csv	Product master (prices, reorder levels)
Dim_Store.csv	Store IDs and names
Dim_Suppliers_20k.csv	Supplier names, quality ratings, lead times

11. Known Issues & Recommendations

Limitations, hardcoded values, and improvement suggestions

11.1 Hardcoded Dataset Path

Issue

base = r"C:\Users\Asus\OneDrive\Desktop\pharma_dash" will fail on any other machine.
This is a development artifact that must be resolved before sharing or deploying the application.

Recommendation: Use environment variable or os.path relative resolution:

```
base = os.environ.get("PHARMA_DATA_DIR",  
    os.path.join(os.path.dirname(os.path.abspath(__file__)), "dataset"))
```

11.2 Product Name Dataset Limitation

The dataset contains only 6 unique medicine name templates (e.g. "Medicine_A", "Medicine_B") shared across 170 unique ProductIDs. This means:

- Any metric based on ProductName.nunique() returns 6, not the real SKU count.
- The bug fixes in v2 (get_expiring_soon_n, get_out_of_stock_by_store) address this by using ProductID for uniqueness checks.
- In production with real data (unique names per SKU), the original .nunique() would have worked correctly.

11.3 GST and Net Profit Estimates

The profitability estimates are approximate:

- GST is fixed at 12% of revenue. Real pharmacy GST in India varies by product category (0%, 5%, 12%, 18%).
- COGS is estimated as QuantitySold × UnitCost. This does not account for returns, shrinkage, or wastage adjustments in the sales data.
- Net Profit = Revenue – COGS – Wastage does not include operating expenses (rent, salaries, utilities).

11.4 AI API Availability

The Groq client uses Llama-3.3-70B via the Groq cloud API. Risks:

- Rate limits: Free tier has token-per-minute limits. High-traffic usage may hit limits.
- API availability: The dashboard degrades gracefully if Groq is unavailable — AI sections show a "not configured" message.
- Context window: The summary_data string grows with richer context in v2. Monitor for token limit issues with large date ranges.

11.5 Recommended Improvements

- Move dataset path to environment variable or config file.
- Add unit tests for bug-fix functions (get_expiring_soon_n, get_out_of_stock_by_store, get_supplier_last_delivery_days).
- Replace fixed 12% GST rate with per-category GST rate table.
- Add a login/authentication layer before deployment (Streamlit does not include auth out of the box).
- Consider connecting to a live database (PostgreSQL / MySQL) instead of static CSVs for real-time inventory data.
- Add export to PDF in addition to Excel for management reports.

12. Glossary

Key terms, abbreviations, and domain concepts

@st.cache_data	Streamlit decorator that memoises a function's return value keyed by its arguments.
Batch	A specific delivery lot of a product identified by BatchNumber, with its own expiry date.
Category dtype	Pandas dtype that stores string columns as integer codes, reducing memory and speeding groupby.
COGS	Cost of Goods Sold — estimated as $\text{QuantitySold} \times \text{UnitCost}$.
DaysToExpiry	Number of days from today until a batch's expiry date. Negative = already expired.
ExpiryStatus	Categorical label: Valid (>90 days), Near Expiry (1–90 days), Expired (<0 days).
Glassmorphism	UI design style using semi-transparent frosted glass backgrounds with soft shadows.
Groq	Cloud inference platform providing fast Llama model serving via API.
GST	Goods and Services Tax — Indian indirect tax applied to pharmacy sales (estimated at 12%).
KPI	Key Performance Indicator — headline metric displayed in a dashboard card.
Llama-3.3-70B	Meta's large language model, 70 billion parameters, used for AI insights generation.
ReorderPoint	Minimum stock level at which a new order should be placed to avoid stockout.
RiskScore	Composite supplier risk metric combining OOS batches, low stock, expiry, delay, and quality.
SafetyStock	Minimum buffer inventory held to absorb demand variability and supply delays.
SKU	Stock Keeping Unit — unique product identifier (ProductID in this dataset).
st.session_state	Streamlit's mechanism for persisting values across reruns within a user session.
Vectorised op	NumPy/Pandas operation that processes an entire array in a single C-level call, faster than Python loops.
Wastage	Estimated value of expired or unsellable stock ($\text{QuantityOnHand} \times \text{UnitCost}$ for expired batches).



PharmaDash Analytics

Version 2.0 | May 2026 | © 2026 PharmaDash Analytics
Documentation generated from *pharmacy_updated.py* source code